

Mitali Bhatia

D20C 04

Blockchain

Experiment No 06

Aim: To create and deploy a Smart Contract and perform transactions using Solidity and Remix IDE.

Theory:

A **smart contract** is a program stored on a blockchain that automatically performs a particular task according to the given rules. In this experiment, a **Voting/Ballot smart contract** is created using **Solidity** and deployed using **Remix IDE**.

1. require Statement

The require statement is used to check whether a condition is true before executing a function.

- If the condition is true, the function continues.
- If the condition is false, the transaction stops and the changes are cancelled.
- It is mainly used for **security and validation**.

Example:

```
require(!sender.voted, "You already voted.");
```

This prevents a person from voting more than once.

2. Mapping

A **mapping** connects a key with a value, similar to a table.

In the Ballot contract:

```
mapping(address => Voter) public voters;
```

Here, each voter's **Ethereum address** is connected to their voting information such as voting status, weight and selected proposal.

3. Storage

Storage is the permanent storage of data on the blockchain.

- Data remains available after the function finishes.
- It is used for important contract data.
- Writing data to storage requires more gas.

Example:

```
Voter storage sender = voters[msg.sender];
```

4. Memory

Memory is temporary storage used while a function is running.

- Data is removed after the function finishes.
- It is cheaper than storage.
- It can be used for temporary data such as function parameters.

Example:

```
constructor(bytes32[] memory proposalNames)
```

5. bytes32

bytes32 is a fixed-size data type that can store up to **32 bytes** of data.

It is used for proposal names because it is more efficient and generally uses less gas than a dynamic string.

The Ballot contract uses:

```
bytes32 name;
```

Thus, the experiment demonstrates how **require statements, mapping, storage, memory and bytes32** are used in a Solidity smart contract.

Code:

```
// SPDX-License-Identifier: MIT                                uint256 indexed candidateId

pragma solidity ^0.8.20;                                       );

contract VotingBallot {                                       constructor() {

    struct Candidate {                                         owner = msg.sender;

        string name;                                         }

        uint256 voteCount;                                     // Add a new candidate

    }                                                         function addCandidate(string memory
                                                                name) public {

        Candidate[] public candidates;                         require(

        mapping(address => bool) public                         msg.sender == owner,

        hasVoted;                                             "Only owner can add candidate."

        address public owner;                                   );

        event CandidateAdded(                                   );

            uint256 indexed candidateId,                       candidates.push(

            string name                                         Candidate({

                                                                name: name,

                                                                voteCount: 0

                                                                })

        );
    event VoteCast(

        address indexed voter,
```

<pre>); emit CandidateAdded(candidates.length - 1, name); } // Vote for a candidate function vote(uint256 candidateId) public { require(!hasVoted[msg.sender], "You have already voted."); require(candidateId < candidates.length, "Invalid candidate."); candidates[candidateId].voteCount += 1; hasVoted[msg.sender] = true; </pre>	<pre> emit VoteCast(msg.sender, candidateId); } // Get candidate name and vote count function getCandidate(uint256 candidateId) public view returns (string memory, uint256) { require(candidateId < candidates.length, "Invalid candidate."); return (candidates[candidateId].name, candidates[candidateId].voteCount </pre>
--	---

);	{
}	require(
// Get total number of candidates	candidates.length > 0,
function getCandidateCount()	"No candidates."
public	);
view	
returns (uint256)	uint256 winningIndex = 0;
{	for (
return candidates.length;	uint256 i = 1;
}	i < candidates.length;
// Find the candidate with the highest votes	i++
function getWinner()	) {
public	if (
view	candidates[i].voteCount >
returns (	candidates[winningIndex].voteCount
string memory winnerName,	) {
uint256 winnerVotes	winningIndex = i;
)	}

```

    }

    candidates[winningIndex].voteCount

    );

    return (

    }

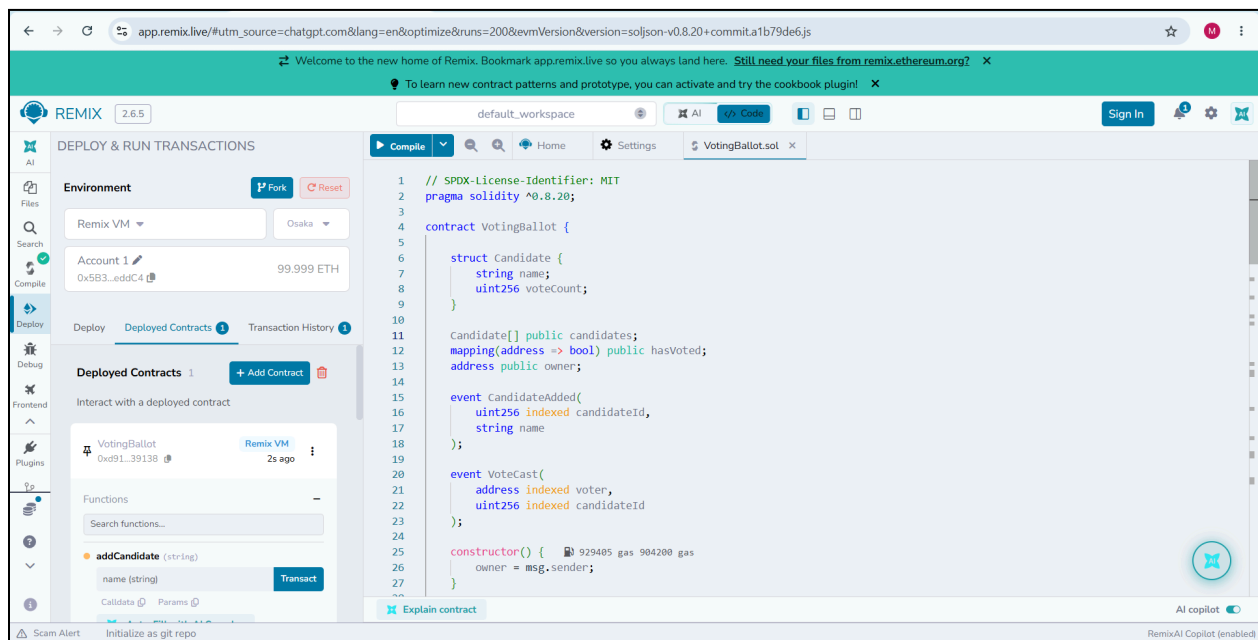
    candidates[winningIndex].name,

    }

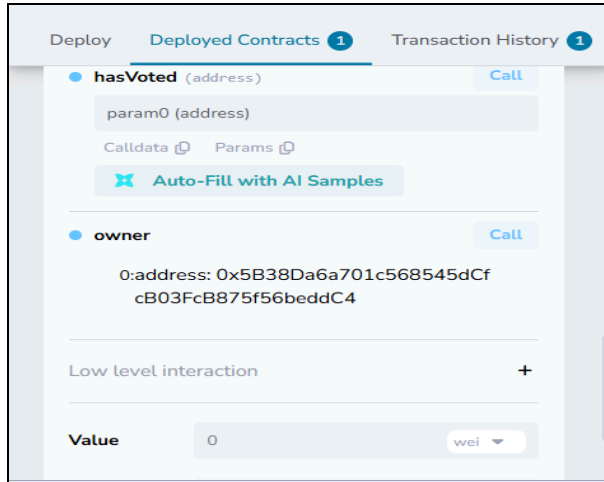
```

OUTPUT:

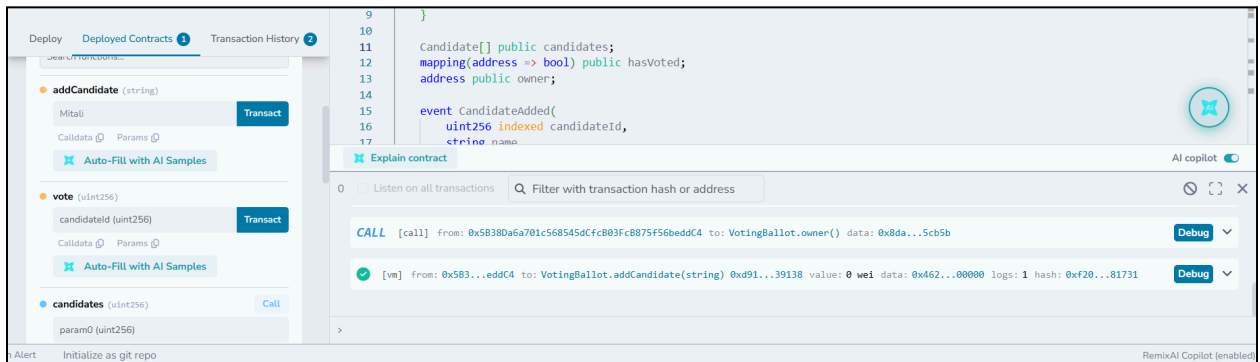
Contract Deployed



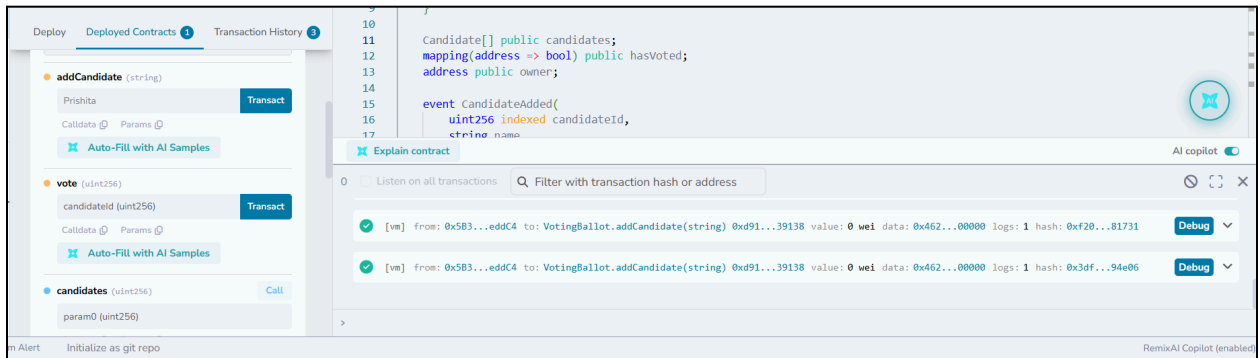
Owner Of the Contract



Added Mitali as candidate



Added Prishita as candidate



Added Vanshika as Candidate

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' panel is active, displaying the 'addCandidate' function with the parameter 'Vanshika'. The 'Transact' button is highlighted. Below it, the 'vote' function is also visible. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'Candidate' struct and the 'addCandidate' function. The 'Explain contract' panel at the bottom shows the transaction details for the 'addCandidate' function, including the transaction hash and the value of 0 wei.

Added Hitesh as candidate

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' panel is active, displaying the 'addCandidate' function with the parameter 'Hitesh'. The 'Transact' button is highlighted. Below it, the 'vote' function is also visible. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'Candidate' struct and the 'addCandidate' function. The 'Explain contract' panel at the bottom shows the transaction details for the 'addCandidate' function, including the transaction hash and the value of 0 wei.

Calling getCandidateCount

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' panel is active, displaying the 'getCandidateCount' function. The 'Call' button is highlighted. Below it, the 'getWinner' and 'hasVoted' functions are also visible. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'Candidate' struct and the 'getCandidateCount' function. The 'Explain contract' panel at the bottom shows the transaction details for the 'getCandidateCount' function, including the transaction hash and the value of 0 wei.

Calling getCandidate on Mitali

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' panel lists 'candidates (uint256)' and 'getCandidate (uint256)'. The 'getCandidate' function is selected, and its parameters are set to '0' and 'Mitali'. The 'Transaction History' panel on the right shows a list of transactions. The first transaction is a 'CALL' from '0x58380a6a701c568545dcfc803fc8875f56beddC4' to 'VotingBallot.getCandidateCount()' with data '0x30a...56347'. The second transaction is a 'CALL' from the same address to 'VotingBallot.getCandidate(uint256)' with data '0x35b...00000'. The 'Explain contract' panel at the bottom shows the Solidity code for the 'VotingBallot' contract, including the 'getCandidate' function.

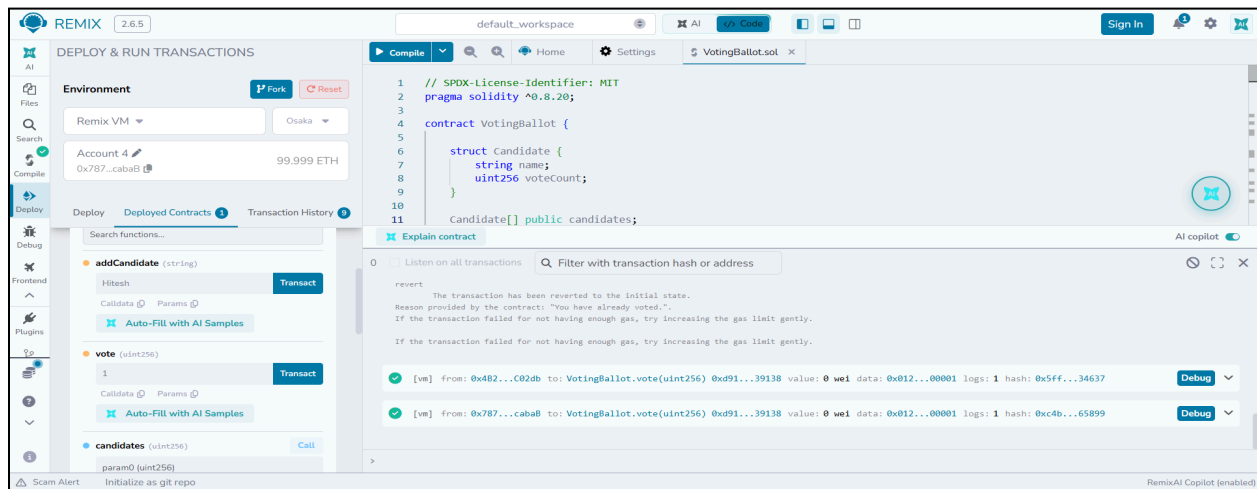
Voting for Mitali from Account 2

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' panel lists 'vote (uint256)', 'candidates (uint256)', and 'getCandidate (uint256)'. The 'vote' function is selected, and its parameter is set to '0'. The 'Transaction History' panel on the right shows a list of transactions. The first transaction is a 'CALL' from '0x58380a6a701c568545dcfc803fc8875f56beddC4' to 'VotingBallot.getCandidate(uint256)' with data '0x35b...00000'. The second transaction is a 'VM' transaction from '0xab8...35cb2' to 'VotingBallot.vote(uint256)' with value '0 wei' and data '0x012...00000'. The 'Explain contract' panel at the bottom shows the Solidity code for the 'VotingBallot' contract, including the 'vote' function.

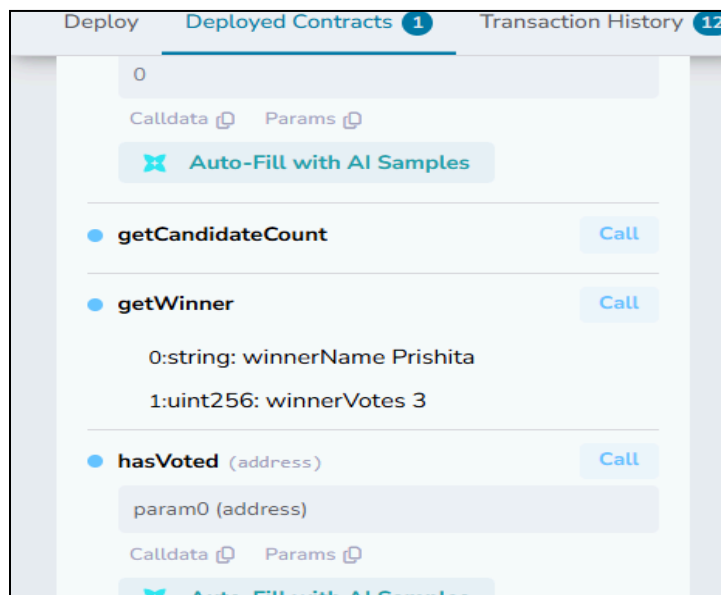
Preventing Double Voting

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' panel lists 'vote (uint256)', 'candidates (uint256)', and 'getCandidate (uint256)'. The 'vote' function is selected, and its parameter is set to '0'. The 'Transaction History' panel on the right shows a list of transactions. The first transaction is a 'CALL' from '0x58380a6a701c568545dcfc803fc8875f56beddC4' to 'VotingBallot.getCandidate(uint256)' with data '0x35b...00000'. The second transaction is a 'VM' transaction from '0xab8...35cb2' to 'VotingBallot.vote(uint256)' with value '0 wei' and data '0x012...00000'. The 'Explain contract' panel at the bottom shows the Solidity code for the 'VotingBallot' contract, including the 'vote' function. The transaction failed with an error: 'Error occurred: revert. Reason provided by the contract: "You have already voted."'. The error message also states: 'The transaction has been reverted to the initial state. If the transaction failed for not having enough gas, try increasing the gas limit gently.. Error occurred: revert. The transaction has been reverted to the initial state.'

Using 3rd , 4th and 7th Account to Vote for Prishita i.e 1



Getting Winner



Conclusion:

In this experiment, a **VotingBallot smart contract** was created using **Solidity** and deployed on the **Remix IDE**. Candidates were added and their votes were recorded using blockchain transactions. The experiment also demonstrated how the `require` statement prevents invalid actions such as **double voting**. Finally, the candidate with the highest number of votes was identified as the winner. Thus, the experiment helped us understand the basic working of a **voting smart contract**, **transactions**, **candidate management**, and **vote validation using blockchain**.